

```
# Load the chain
sudo bfccli chain load --from-str "
    chain my_xdp_chain BF_HOOK_XDP ACCEPT
    rule
        ip4.proto eq icmp
        counter
        DROP"

# Attach to the interface (use `ip a` to find ifindex)
sudo bfccli chain attach --name my_xdp_chain --option ifindex=2

# Try pinging now

# Reset
sudo bfccli chain flush -n my_xdp_chain
```

You can try pinging from another machine before and after attaching the chain to see that it actually works.

bpfilter is meant to be full-fledged, supporting filtering at various levels of the network stack. This includes XDP (eXpress Data Path), an eBPF-based kernel feature that lets packets to be filtered at the NIC driver level (with or without hardware support). This way, filtering happens before packets enter the kernel network stack, making it a high-performance solution. This is why XDP is often picked for DDoS mitigation.

Visit bpfilter.io and their GitHub repository for more information; installation is easy, but out of scope of this article. There is also a project called xdp-filter, which is a packet filter that focuses exclusively on XDP.

Another area where ready-made eBPF tools are available is observability. bpfftrace is a tool that lets us write small programs to monitor kernel events and collect statistics. It accepts source programs in an AWK-like language and converts them to eBPF programs internally. The language was inspired by the D language used by DTrace, another UNIX kernel tracing solution.

Here is an example command that logs every time some process removes a directory:

```
bpfftrace -e 'tracepoint:syscalls:sys_enter_rmdir {
    printf("%s removed some directory\n", comm); }'
```

After loading our tracing program like this, create an empty directory and remove it (either using GUI or by issuing commands in a different terminal). You can see that the directory removal is being logged. The process can be terminated using *Ctrl+C* as usual.

How eBPF works

So far, our understanding is that eBPF is a technology that lets us inject custom code into the operating system kernel on the go. Let's go a bit deeper than that.

An eBPF program is essentially an executable binary in the ELF format (similar to any other compiled program you run on Linux-based systems). The key difference is that it contains instructions in the eBPF bytecode instead of actual machine code. One could write eBPF programs in Assembly, C, etc, and get them compiled using Clang or any other supported compiler.

Now we need a user space 'loader program', which sends the eBPF program to the kernel by issuing a `bpf()` system call. There are libraries available in languages including C, Go, and Python to make this far easier. The loader program can also stick around and communicate with the eBPF program to collect data and perform complex operations.

Once the program is sent to the kernel, it is inspected by a key component called 'the eBPF verifier', which makes sure the program is free of infinite loops and illegal operations. If the program passes, it gets compiled to the native machine code (say `x86_64`) by a JIT compiler. After that, the program is ready for direct execution in the kernel space. This whole process is given in Figure 1.

eBPF execution is event driven. So, when an eBPF program gets executed depends on what hook it was attached to. Some example hooks include XDP (useful for high-performance filtering of network packets) and syscall tracepoints (triggered for system calls like `open()`, `unlink()`, `fork()`, etc).

Another key concept in the eBPF infrastructure is *maps*. These are data structures like arrays, queues, hash maps, etc, shared between eBPF programs and their loader programs for communication between the kernel space and the user space.

Programming and deploying eBPF

We just saw that one has to write two programs – a user space loader and the actual eBPF program – if one wants to go deeper than high-level tools like bpfftrace or bpfilter. While both can be written in high-level languages, doing so without depending on some framework would be very difficult.

Some popular eBPF development frameworks/methods are BCC (BPF Compiler Collection), libbpf-bootstrap, and eunomia-bpf. We'll pick libbpf-bootstrap for this article as it's a good starting point while still being portable and supported by some of the kernel eBPF maintainers.

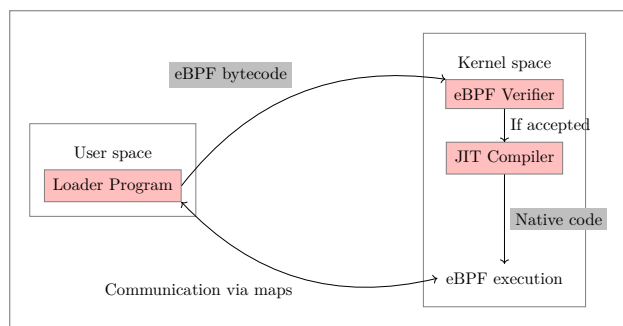


Figure 1: An overview of how eBPF programs get loaded into the Linux kernel